

End-to-End Efficiency of the Packed Projected W1X1/N1 ImageNet ViT

Reviewer-response insert: architecture, precision allocation, projection cost, latency, throughput, memory, energy, full-precision ablation, and scaling

Reviewer question. “Provide actual efficiency measurements (latency, throughput, memory, energy, or bit-ops) for a representative ViT, accounting for projection expansion and all full-precision modules. This is the decisive item.”

Response in one sentence. For a complete 12-block, 224×224 ImageNet ViT on an H100 at batch 2, the packed structured W1X1/N1 deployment target is **1.724× faster**, delivers **1.724× higher throughput**, consumes **30.6% less gross device energy per image**, and uses **9.52× less persistent model state** than a TorchInductor-compiled BF16 model with the same topology; the timing includes the projection, attention, MLPs, norms, residuals, patch embedding, and classifier.

Evaluated topology and precision map. The model has 197 tokens (196 patches plus class token), width $d = 384$, 12 Transformer blocks, six 64-wide heads, a $384 \rightarrow 1536 \rightarrow 384$ MLP, and 1,000 output classes. Its 72 body matrices are $W_Q, W_K, W_V, W_O, W_{up}, W_{down}$ in every block. The projection factor is $N/d = 1$; hence there is no code-length expansion in this experiment.

Component	Stored precision	Runtime treatment
72 body matrices	$Q(GW)$, one bit/coordinate	$Q(Gx)$ versus $Q(GW)$ through XOR-popcount and FP32 row-gain epilogue
Projection G	four shared int8 sign diagonals, 3,840 bytes total	one implicit block-32 Hadamard pass; dense G is never created
Projected activations	not persistent	sign-packed and consumed on chip; expanded Gx is never written to global memory
Residual stream and attention	transient FP16	Flash/SDPA attention; FP16 block-to-block residual
Retained FP state	FP16/FP32	patch embed, token/position state, norms, gains, RPreLU/residual/attention scales, final norm/head

Complete ImageNet ViT, H100, batch 2

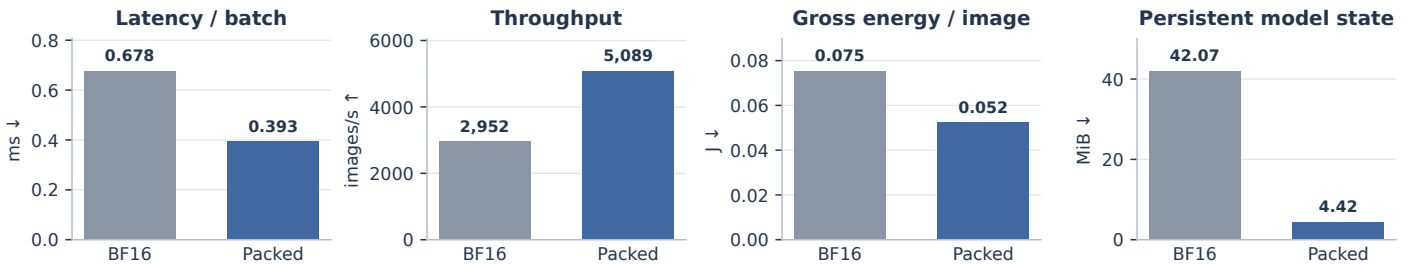


Figure 1. Measured complete-model H100 result. Lower is better for latency, energy, and state; higher is better for throughput.

Headline protocol and result. NVIDIA H100 80 GB HBM3; PyTorch 2.12.0+cu126; CUDA 12.6; preloaded deterministic inputs; 20 warm-ups; 200 paired CUDA-event samples; static-shape CUDA Graphs; separate eight-second NVML loops under a shared measured idle power. The BF16 comparator uses the same 12×384 topology, fused QKV, Flash/SDPA, and TorchInductor. Median latency is 0.677536 versus 0.393024 ms/batch; throughput is 2,951.9 versus 5,088.7 images/s; gross energy is 0.075477 versus 0.052357 J/image (1.442× efficiency); idle-adjusted energy is 0.034337 versus 0.028547 J/image (1.203×).

Accuracy boundary. The available trained checkpoint uses dense Haar-orthogonal projections and reports 56.188% top-1 / 79.612% top-5. Its exact packed loader has zero observed logit error against the unpacked checkpoint operator. The headline performance path instead uses the hardware-oriented one-pass Hadamard recipe and therefore *is not an accuracy claim for that old checkpoint*; it is the deployment target that must be used during training. We deliberately keep systems evidence and accuracy evidence separate rather than silently changing G .

Where the memory reduction actually comes from

Reviewer question. “If random projections are regenerated, are the original FP weights and activations still stored and communicated? Would that not be larger than full precision, especially if activation traffic dominates?”

The short answer is **no for the weights, partially yes for the residual activations, and no dense projection is regenerated**. Training and deployment must be distinguished:

- **During QAT**, latent FP weights and optimizer state may be retained. We do *not* claim a training-memory reduction.
- **During deployment**, latent W is absent. Each body matrix is represented only by its packed $Q(GW)$ bits plus a row gain. The four shared sign diagonals specify two structured transforms ($d = 384$ and 1536); no dense G is stored or regenerated.
- **The residual stream remains FP16**. We do not claim that every activation is persistently one bit. Instead, $Q(Gx)$ is formed and consumed inside the fused kernel, and selected 384- and 1,536-wide intermediates remain in registers/shared memory.

Let $P_{\text{body}} = 21,233,664$ be the number of binary body weights. The dense BF16 body alone requires

$$M_{\text{body}}^{\text{BF16}} = 2P_{\text{body}} = 42,467,328 \text{ bytes} = 40.500 \text{ MiB},$$

whereas its one-bit codes require $P_{\text{body}}/8 = 2,654,208$ bytes (2.531 MiB). After adding *all* gains, retained FP modules, runtime constants, and the 3,840-byte projection recipe, the measured deployed state is

$$M_{\text{BF16}} = 42.067 \text{ MiB}, \quad M_{\text{packed}} = 4.419 \text{ MiB}, \quad M_{\text{BF16}}/M_{\text{packed}} = 9.52.$$

This is below $16\times$ because full-precision boundaries and gains are included.

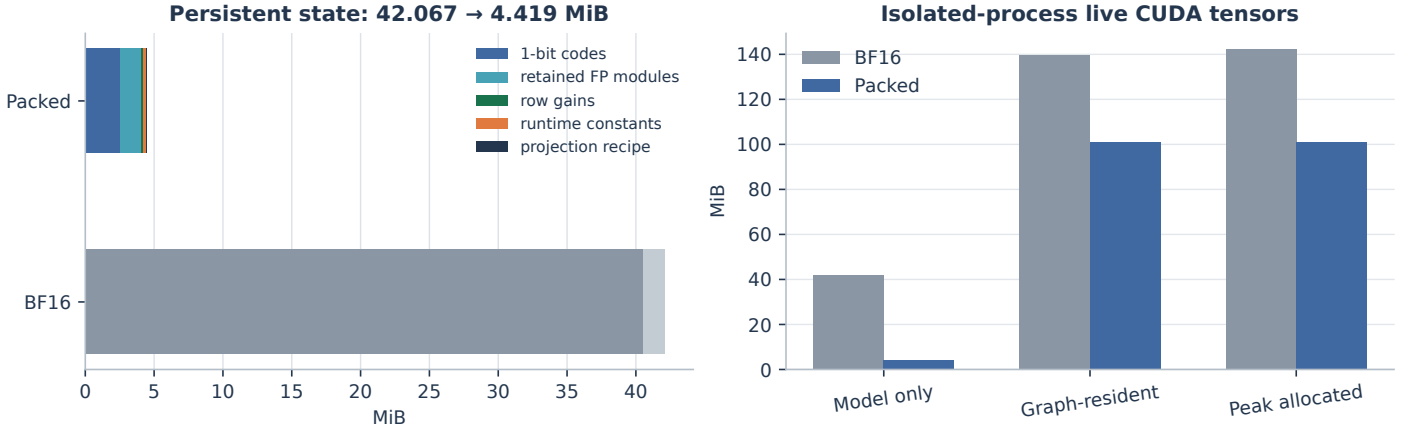


Figure 2. Left: exact unique-storage accounting. Right: isolated-process live CUDA tensor allocations at batch 2 after compilation and CUDA-Graph capture. CUDA-context memory, common to both candidates, is excluded.

Actual total runtime allocation, not only file size. In isolated processes, model+input allocation is 43.547 MiB for BF16 and 5.029 MiB for packed. After compilation and static graph capture, live CUDA tensors are 139.548 and 101.030 MiB; measured peak allocation is 142.191 and 101.036 MiB. Thus fixed compiler/graph workspaces reduce the whole-process advantage to $1.38\text{--}1.41\times$, even though persistent state is $9.52\times$ smaller. The serialized packed checkpoint is 4.268 MiB and contains no latent body weights, optimizer state, or dense G .

What happens to x . One FP16 residual tensor is only $197 \times 384 \times 2 = 151,296$ bytes (0.144 MiB) per image and is reused sequentially; it is not stored once per layer at inference. Q/K/V and attention context remain FP16 because attention is retained. However, the fused post-attention/MLP kernel avoids global materialization of the 384-wide attention-output boundary and the 1,536-wide MLP hidden state—about 0.721 MiB of writes per image per block, plus the corresponding reads. The binary projected activation is ballot-packed and consumed within the kernel.

Why “weights are loaded once for the entire batch” is not generally correct. Weights move through registers, shared memory, L1, L2, and HBM; reuse depends on shape, cache capacity, and batch size. A layer’s weights may be reused across the token/batch dimension, but a model larger than cache still streams layer state from HBM each inference. Our current 42.067 MiB BF16 model can largely fit in the H100’s 50 MB L2, which is unusually favorable to the dense baseline. Figure 3 directly shows that increasing batch amortizes dense weight traffic and eventually reverses the speedup.

Actual execution cost and the full-precision ablation

Reviewer question. “Which Transformer modules are quantized, and how much do retained full-precision components contribute? Does the gain survive realistic projection and FP accounting?”

Measured CUDA-time attribution. A 50-iteration CUDA trace of the same compiled batch-2 path totals 0.390696 ms/call, matching the 0.393024 ms headline. Projected low-bit kernels account for 57.57%; retained Flash attention accounts for 27.63%; the FP16 patch convolution for 12.33%; and the classifier, final norm, token boundary, and layout for the remaining 2.47%. The projection, packing, popcount, gain, normalization, activation, and residual epilogues are already inside the two blue low-bit categories below.

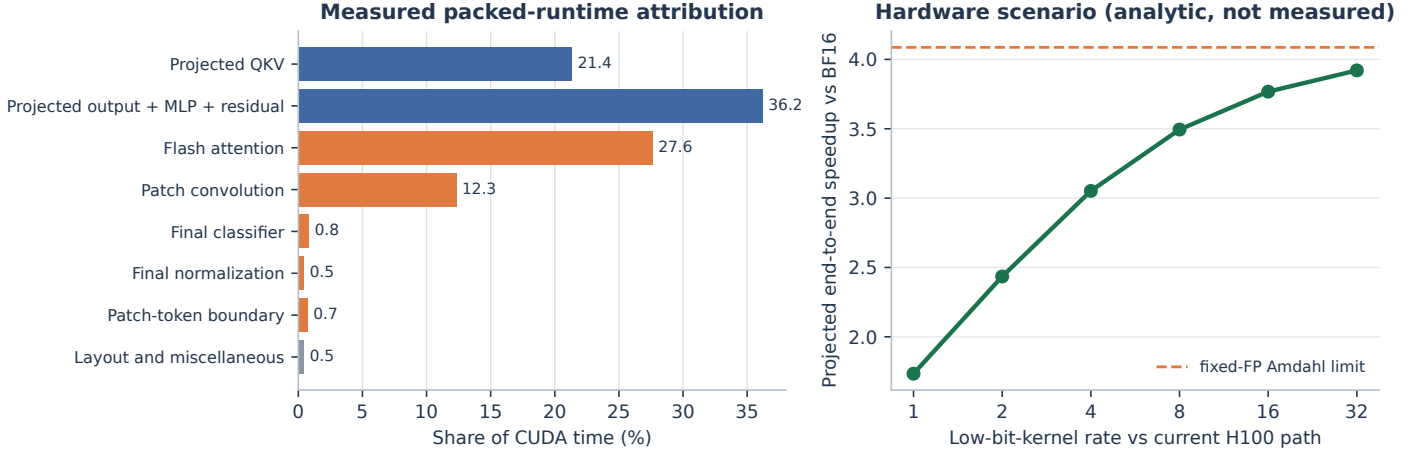


Figure 3. Left: measured packed-runtime CUDA attribution. Right: latency-derived Amdahl scenario for a native low-bit engine; the curve is an analytic projection, not an H100 measurement.

Per-image operation class	Count	Precision / implementation
Body linear pair products	4.183 billion	one-bit signs; 130.72 million 32-bit XNOR-popcount word terms
Structured projections	31.77 million	add/sub/sign operations; one block-32 Hadamard pass
Attention QK^T and AV	29.81 million	retained FP16 Flash/SDPA
Patch embedding	57.80 million	MACs
		retained FP16 convolution
Classifier	0.384 million	MACs
		retained FP16 GEMM

Why the implementation is genuinely low-bit-aware. One normalized/projected activation is reused for Q/K/V. Width-384 and width-1,536 projections use a shared seeded block-Hadamard recipe. Custom CUDA operators fuse RMSNorm/centering, Hadamard, sign ballot, XNOR-popcount, row gains, RPRReLU, residual scaling, and residual addition. The attention-output projection and both MLP linears execute in one cross-sublayer kernel, so the intermediate activation does not make a round trip to HBM. The first MLP activation projection is register-resident. Proper `torch.library` schemas and fake/meta kernels leave one Dynamo graph and zero graph breaks; both candidates use CUDA Graph replay.

Batch/cache ablation. The packed path is faster at batches 1, 2, 4, 6, and 8, with speedups of 1.542 $\times$, 1.724 $\times$, 1.513 $\times$, 1.202 $\times$, and 1.182 $\times$. The crossover lies between batch 8 and 16: at 16 and 32, the speed ratios are 0.894 $\times$ and 0.776 $\times$, because dense weights are heavily reused and BF16 Tensor Cores become well utilized.

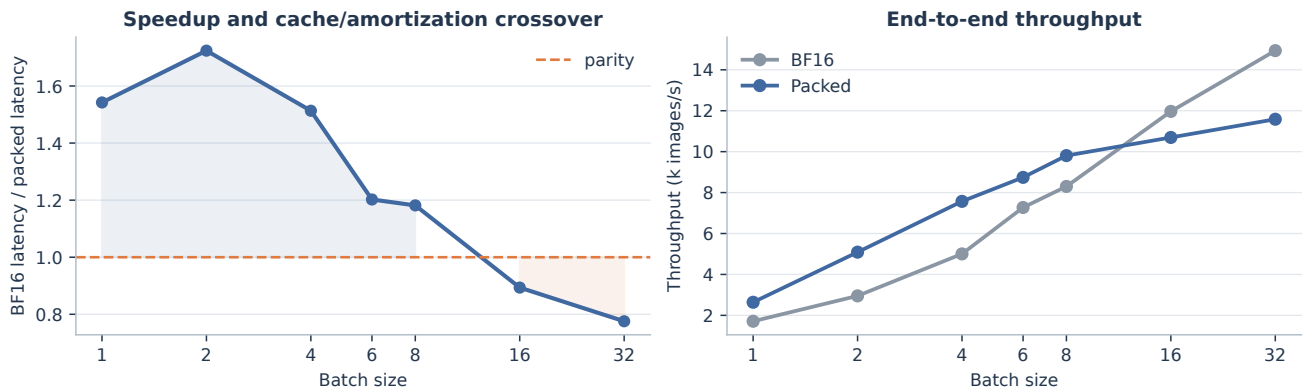


Figure 4. Latest-kernel sweep. The low-bit advantage is an operating-region result, not a claim of universal speedup.

Why H100 is conservative, and what changes at larger scale

The hardware disadvantage is concrete. H100 is designed to make the comparator exceptionally strong: mature cuBLAS/cuDNN/Inductor kernels and fourth-generation Tensor Cores provide very high BF16/FP16/FP8 throughput, while the deployed one-bit path uses shape-specialized CUDA integer operations rather than an equally general, high-throughput binary Tensor Core. H100 exposes fixed binary MMA primitives, but their tile/operand constraints were slower for this 197-token, width-384 fused workload; native BMMA, paired-token reuse, 384/512-thread MLPs, direct scattered Q/K stores, and a fused scalar final head were all measured and rejected. Moreover, the entire 42.067 MiB dense state can reside largely in H100’s 50 MB L2 after warm-up.¹

What should improve at larger model scale. The present ~ 22 M-parameter ViT is a stringent cache-resident case. Representative ViT-B/16 (~ 86 M parameters, ~ 172 MiB in BF16) and ViT-L/16 (~ 307 M, ~ 614 MiB) exceed H100 L2 by wide margins.² At small batch, such models must stream much more dense layer state from HBM, while packed one-bit bodies can fit substantially more of the working set in cache and move roughly one-sixteenth as many body-weight bytes. Therefore the memory-bound regime should persist to larger batches and the crossover should move right. This is a roofline expectation, not a substituted measurement: larger-model results must report the same batch sweep because sufficiently large batches again amortize weight traffic.

Expected outcome on hardware designed for extreme low-bit compute. A suitable accelerator would provide native packed XNOR–popcount matrix units, packed activation SRAM, an implicit Hadamard/sign front end, and fused FP gain/residual epilogues. The current H100 trace partitions 0.390696 ms into 0.2249 ms low-bit kernels and 0.1658 ms retained FP/fixed work. If a native engine accelerated only the low-bit portion by $8\times$ or $16\times$, the transparent Amdahl estimates are

$$T(8) = 0.1658 + 0.2249/8 = 0.1939 \text{ ms}, \quad S_{e2e} = 3.49\times,$$

$$T(16) = 0.1658 + 0.2249/16 = 0.1798 \text{ ms}, \quad S_{e2e} = 3.77\times.$$

The present precision allocation has a $4.09\times$ hard latency ceiling unless patch embedding, attention, and the final boundary are also quantized/fused. These are *latency-derived scenarios*, not fabricated ASIC measurements. Likewise, bit count alone is not converted into an energy claim; the defensible energy result remains the measured H100 reduction of 30.6% gross (16.9% idle-adjusted) per image. Low-bit-friendly hardware should improve energy primarily by eliminating operand movement and replacing FP multipliers with packed Boolean/popcount datapaths.

Claim boundary and reproducibility.

- The document establishes a representative ImageNet ViT systems result; it does not relabel this as an NLP measurement.
- The BF16 comparator has identical topology and execution precision but random body values because the compact artifact intentionally contains no latent FP body. Weight values do not change these static-shape timings.
- The exact packed checkpoint path preserves the trained operator; the faster structured path is a separately trainable deployment target. A final accuracy–efficiency claim requires training with that exact one-pass structured G .
- Canonical raw reports, batch-sweep CSV, isolated-memory audit, profiler attribution, plotting source, and build instructions accompany this PDF.

¹NVIDIA H100 architecture and 50 MB L2: <https://resources.nvidia.com/en-us-tensor-core/nvidia-hopper-architecture-whitepaper>.

²Vision Transformer architecture: A. Dosovitskiy et al., “An Image is Worth 16×16 Words,” ICLR 2021, <https://arxiv.org/abs/2010.11929>.

Expansion-factor cost and the structured projection recipe

Reviewer question. “Standard layers compute a p -dimensional inner product, whereas Angular Layers also compute Gx and Gw with $G \in \mathbb{R}^{N \times p}$ and $N = \nu p$. How does this affect training and inference cost for $\nu \in \{1, \dots, 4\}$? Were practical slowdowns observed?”

Matrix-free random structured Hadamard. For block width $b = 32$ and frame f , the deployed recipe is

$$G_f x = \text{vec} \left(\frac{1}{\sqrt{b}} H_b D_{2,f} H_b D_{1,f} x^{(j)} \right)_{j=1}^{p/b}, \quad G = [G_1^\top, \dots, G_\nu^\top]^\top, \quad N = \nu p,$$

where $H_b H_b^\top = bI$ and each diagonal $D_{r,f}$ contains independent Rademacher signs regenerated from versioned SplitMix64 seed streams. The ViT-S body has only two input dimensions ($p = 384$ and 1536), so it shares two recipes across all 72 body matrices. No dense G or latent body weight is present at deployment. Packed $Q(G_f W)$ is computed before inference; only $Q(G_f x)$ is formed online, and Q/K/V reuse it.

The online transform uses $O(2\nu p \log_2 32)$ add/sub operations per token and the packed contraction contains νmp sign-pair products. Weight-code storage is νmp bits. These counts grow linearly in ν ; the complete-model measurements below show how much that linear work contributes after fixed FP attention, patch embedding, norms, residuals, and classifier are included.

Complete training measurement. Forward, cross-entropy backward, and one SGD update for the complete 12-block 224×224 ViT-S at batch 2:

ν	step (ms)	images/s	change vs $\nu = 1$	peak allocated (MiB)
1	16.510	121.14	—	208.0
2	16.693	119.81	+1.1%	208.8
3	17.498	114.30	+6.0%	208.8
4	17.932	111.53	+8.6%	208.2

The compiled BF16 comparator is 7.535 ms, so measured angular QAT is $2.19\times$ slower at $\nu = 1$ and $2.38\times$ at $\nu = 4$. Expansion itself adds only 8.6% because the analytical backward and retained FP operations do not expand.

Complete packed-inference expansion measurement. All factors use the same portable packed implementation, precomputed $Q(GW)$, compiler-visible CUDA operators, CUDA Graph replay, 25 warm-ups, 400 CUDA-event samples, and eight-second NVML loops at batch 2. The architecture-specific, cross-sublayer-fused $\nu = 1$ headline remains 0.393024 ms/batch; the portable $\nu = 1$ row below is deliberately less fused and is only the denominator for a controlled expansion sweep.

ν	latency (ms/batch)	vs $\nu = 1$	images/s	gross J/image	persistent MiB
1	1.089584	1.000×	1,835.6	0.125620	4.258
2	1.318784	1.210×	1,516.5	0.169080	6.789
3	1.575168	1.446×	1,269.7	0.214700	9.320
4	1.878624	1.724×	1,064.6	0.263485	11.851

Every factor exactly matches its QAT hard forward (maximum logit difference 0; 100% top-1 agreement). From $\nu = 1$ to 4, latency rises 72.4%, gross energy/image rises 109.7%, and code storage rises exactly $4\times$; total persistent state rises $2.78\times$ because the retained FP boundaries do not expand.

Why our default is $\nu = 1$. Larger ν reduces Monte Carlo variance but is not the most efficient accuracy knob observed in our experiments. At fixed $N = p$, increasing w/x quantizer resolution generally preserves more information per added bit and avoids repeating the Hadamard transform. We do not call higher bits literally free: QAT keeps the same matrix dimensions and can have similar wall time, but a generic bit-serial runtime needs up to $b_w b_x$ binary planes and stores b_w weight planes. Thus W2X2/N1 and W1X1/N4 both expose four binary pair-product planes, while only the latter repeats the activation projection four times. This supports $\nu = 1$ as the practical default, with bit-width increased only when the accuracy target requires it.

Claim boundary. The aggressively fused $\nu = 1$ runtime remains the deployment headline. The portable factors 1–4 path is a constant-implementation systems ablation, not a substitute accuracy checkpoint. A $\nu > 1$ accuracy claim requires training with those exact independent structured frames.

Canonical artifact. `best_inference.pth`, 4,475,370 bytes (4.268 MiB), SHA-256 511b760b2f617cf9...06c787a.

Headline report. `runtime/results/headline_b2_reviewer_final.json`.